



PROBLEM SESSION 8

Pipelining

คณะผู้จัดทำ

กันต์ธีร์	ดวงมณี	67070501003
นัธววัฒน์	ปริมสิริคุณาวุฒิ	67070501027
วิศิษฐ์	สุวรรณเนา	67070501042
ศุภวิชญ์	มารยาท	67070501045
พลวริษฐ์	วัฒนเหมรัตน์	67070501067

เสนอ

ผศ. ราชวิช สโรชวิกสิต

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

CPE223 สถาปัตยกรรมคอมพิวเตอร์ (Computer Architectures)

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ภาคเรียนที่ 2 ปีการศึกษา 2568

Introduction

To improve execution throughput, most modern processors deploy pipeline which will overlap stages of executions resulting in many instructions can be in a pipeline at the same time. However, the efficiency of a processor will be degraded due to several types of hazards that could occur in the sequence of instructions and number of control lines and executing unit. Data hazard is one of the hazards that might occur easily in simple programming. We are going to take a look at C codes and compile them into assembly. Then we will identify the hazards and suggest the way to resolve them with the techniques we study in class.

Follow these instructions:

1. You will download a bundle of C source codes announced in class. Extract the file you will see three source files: pipeline_01.c pipeline_02.c and pipeline_03.c.
2. Compile each file with gnu toolchain gcc to generate assembly files with -S flag. ***arm-none-eabi-gcc -S pipeline_01.c -o pipeline_01.s*** or alternatively you can use <https://godbolt.org/>.

Question

Examine each assembly file and identify hazards in those files. Create a report discussing how you can avoid those hazards in C level and assembly level.

Pipeline 1

pipeline_01.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a,b,c,d,e;
6
7      a = 10;
8      c = a+30;
9      b = 20;
10     d = b+20;
11     e = a+b+c+d;
12
13     return e;
14 }
```

pipeline_01.s

```
1      .cpu arm7tdmi
2      .arch armv4t
3      .fpu softvfp
4      .eabi_attribute 20, 1
5      .eabi_attribute 21, 1
6      .eabi_attribute 23, 3
7      .eabi_attribute 24, 1
8      .eabi_attribute 25, 1
9      .eabi_attribute 26, 1
10     .eabi_attribute 30, 6
11     .eabi_attribute 34, 0
12     .eabi_attribute 18, 4
13     .file "pipeline_01.c"
14     .text
15     .align 2
16     .global main
17     .syntax unified
18     .arm
19     .type main, %function
20 main:
21     @ Function supports interworking.
22     @ args = 0, pretend = 0, frame = 24
23     @ frame_needed = 1, uses_anonymous_args = 0
24     @ link register save eliminated.
25     str fp, [sp, #-4]!
26     add fp, sp, #0
27     sub sp, sp, #28
28     mov r3, #10
29     str r3, [fp, #-8]
30     ldr r3, [fp, #-8]
31     add r3, r3, #30
32     str r3, [fp, #-12]
33     mov r3, #20
34     str r3, [fp, #-16]
35     ldr r3, [fp, #-16]
36     add r3, r3, #20
37     str r3, [fp, #-20]
38     ldr r2, [fp, #-8]
39     ldr r3, [fp, #-16]
40     add r2, r2, r3
41     ldr r3, [fp, #-12]
42     add r3, r2, r3
43     ldr r2, [fp, #-20]
44     add r3, r2, r3
45     str r3, [fp, #-24]
46     ldr r3, [fp, #-24]
47     mov r0, r3
48     add sp, fp, #0
49     @ sp needed
50     ldr fp, [sp], #4
51     bx lr
52     .size main, .-main
```

ใน Pipeline 1 จะเกิด **Data Hazard** ประเภท **Read-After-Write (RAW)** หรือ **Data Dependency** เนื่องจากในบางคำสั่งจำเป็นต้องใช้ข้อมูล (Source Operand) ที่เพิ่งถูกประมวลผลโดยคำสั่งก่อนหน้านี้ทันที (ตัวอย่างเช่น การนำค่า 'a' ไปใช้ต่อใน 'c' หรือการใช้ค่า 'd' ในการคำนวณ 'e' ทั้งที่ข้อมูลยังไม่ผ่านขั้นตอน Write Back) ทำให้ข้อมูลยังไม่พร้อมใช้งาน หากไม่มีการจัดการ CPU จะต้องทำการหยุดรอ (**Stall**) หรือ Insert Bubble เพื่อรอให้ข้อมูลพร้อมก่อน ส่งผลให้เสีย Clock Cycle โดยเปล่าประโยชน์

เราสามารถแก้ไขเพื่อลดจำนวน Stall ได้โดยใช้วิธีทาง Hardware เรียกว่า **Operand Forwarding** (หรือ Bypassing) เพื่อส่งผลลัพธ์จากขั้นตอน Execution ของคำสั่งก่อนหน้า ข้ามกลับมายังคำสั่งปัจจุบันทันทีโดยไม่ต้องรอเขียนลง Register หรือใช้วิธีทาง Software ผ่าน **Smart Compiler** โดยทำการจัดเรียงลำดับคำสั่งใหม่ (**Instruction Reordering**) นำคำสั่งที่ไม่เกี่ยวข้องกันมาคั่นกลางเพื่อทิ้งช่วงเวลาให้ข้อมูลพร้อมใช้งาน ช่วยลดปัญหา Data Dependency ที่อยู่ติดกันได้

pipeline_01_fixed.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a,b,c,d,e;
6
7      // แก้ไข Data Hazard โดยการ Reordering (สลับลำดับคำสั่ง)
8      // นำคำสั่งที่ไม่เกี่ยวข้องกันมาแทรกระหว่างคำสั่งที่มี Dependency
9
10     a = 10;
11     b = 20;      // ย้าย b=20 มาแทรก เพื่อให้ a มีเวลาเขียนลง Register ก่อนถูกเรียกใช้ใน c
12     c = a+30;
13     d = b+20;    // d รอ b ซึ่งทำเสร็จไปแล้วข้างบน
14
15     e = a+b+c+d; // ถึงตรงนี้ ข้อมูลทุกตัวพร้อมหมดแล้ว
16
17     return e;
18 }
```

Pipeline 2

pipeline_02.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a;
6      int b[] = {80,70,60,50};
7      int *c;
8      c=b+2;
9      scanf("%d",&a);
10     if(a>100) printf("Error\n");
11     else if(a>80) printf("A\n");
12     else if(a>b[1]) printf("B\n");
13     else if(a>*c) printf("C\n");
14     else if(b[3] <= a) printf("D\n");
15     else if(a>=0) printf("F\n");
16     else printf("Error\n");
17
18     return 0;
19 }
```

```
97 .L6:
98     ldr r2, [fp, #-16]
99     ldr r3, [fp, #-12]
100     cmp r2, r3
101     bgt .L7
102     ldr r0, .L10+24
103     bl puts
104     b .L3
105 .L7:
106     ldr r3, [fp, #-12]
107     cmp r3, #0
108     blt .L8
109     ldr r0, .L10+28
110     bl puts
111     b .L3
112 .L8:
113     ldr r0, .L10+8
114     bl puts
115 .L3:
116     mov r3, #0
117     mov r0, r3
118     sub sp, fp, #4
119     @ sp needed
120     pop {fp, lr}
121     bx lr
122 .L11:
123     .align 2
124 .L10:
125     .word .LC0
126     .word .LC1
127     .word .LC2
128     .word .LC3
129     .word .LC4
130     .word .LC5
131     .word .LC6
132     .word .LC7
133     .size main, .-main
```

pipeline_02.s

```
1  .cpu arm7tdmi
2  .arch armv4t
3  .fpu softvfp
4  .eabi_attribute 20, 1
5  .eabi_attribute 21, 1
6  .eabi_attribute 23, 3
7  .eabi_attribute 24, 1
8  .eabi_attribute 25, 1
9  .eabi_attribute 26, 1
10 .eabi_attribute 30, 6
11 .eabi_attribute 34, 0
12 .eabi_attribute 18, 4
13 .file "pipeline_02.c"
14 .text
15 .section .rodata
16 .align 2
17 .LC1:
18     .ascii "%d\000"
19     .align 2
20 .LC2:
21     .ascii "Error\000"
22     .align 2
23 .LC3:
24     .ascii "A\000"
25     .align 2
26 .LC4:
27     .ascii "B\000"
28     .align 2
29 .LC5:
30     .ascii "C\000"
31     .align 2
32 .LC6:
33     .ascii "D\000"
34     .align 2
35 .LC7:
36     .ascii "F\000"
37     .align 2
38 .LC0:
39     .word 80
40     .word 70
41     .word 60
42     .word 50
43     .text
44     .align 2
45     .global main
46     .syntax unified
47     .arm
48     .type main, %function
```

```
49 main:
50     @ Function supports interworking.
51     @ args = 0, pretend = 0, frame = 24
52     @ frame_needed = 1, uses_anonymous_args = 0
53     push {fp, lr}
54     add fp, sp, #4
55     sub sp, sp, #24
56     ldr r3, .L10
57     sub ip, fp, #28
58     ldm r3, {r0, r1, r2, r3}
59     stm ip, {r0, r1, r2, r3}
60     sub r3, fp, #28
61     add r3, r3, #8
62     str r3, [fp, #-8]
63     sub r3, fp, #12
64     mov r1, r3
65     ldr r0, .L10+4
66     bl scanf
67     ldr r3, [fp, #-12]
68     cmp r3, #100
69     ble .L2
70     ldr r0, .L10+8
71     bl puts
72     b .L3
73 .L2:
74     ldr r3, [fp, #-12]
75     cmp r3, #80
76     ble .L4
77     ldr r0, .L10+12
78     bl puts
79     b .L3
80 .L4:
81     ldr r2, [fp, #-24]
82     ldr r3, [fp, #-12]
83     cmp r2, r3
84     bge .L5
85     ldr r0, .L10+16
86     bl puts
87     b .L3
88 .L5:
89     ldr r3, [fp, #-8]
90     ldr r2, [r3]
91     ldr r3, [fp, #-12]
92     cmp r2, r3
93     bge .L6
94     ldr r0, .L10+20
95     bl puts
96     b .L3
```

ใน Pipeline 2 จะเกิด **Control Hazard** (หรือ Branch Hazard) เนื่องจากในโปรแกรมมีคำสั่งควบคุมทิศทางแบบเงื่อนไข (**Conditional Branch**) หรือคำสั่ง if-clause ซึ่งทำให้ Pipeline ไม่สามารถทราบล่วงหน้าได้ว่าจะต้อง Fetch คำสั่งใดเข้ามาประมวลผลต่อ (Next Program Counter is unknown) จนกว่าการประมวลผลในขั้นตอน Execute ของคำสั่งเปรียบเทียบจะเสร็จสิ้น หาก CPU "ทำนายผิด" ดึงคำสั่งผิดทางเข้ามา จะต้องทำการล้างข้อมูลใน Pipeline ทั้งหมด (**Pipeline Flush**) ทำให้เกิดความสูญเสียเวลาที่เรียกว่า **Branch Penalty**

แนวทางการแก้ไขสามารถทำได้โดยใช้เทคนิค **Branch Prediction** ซึ่งจะมีทั้งรูปแบบ Static (ทำนายตายตัว) และ Dynamic (ทำนายโดยดูประวัติ **History Table** ของโปรแกรม) เพื่อให้ CPU คาดเดาทิศทางได้แม่นยำขึ้น หรือใช้วิธี **Delayed Branch** โดย Compiler จะนำคำสั่งที่ปลอดภัย (Safe Instruction) และต้องทำแน่นอนไม่ว่าผลเงื่อนไขจะเป็นอย่างไร มาวางต่อท้ายคำสั่ง Branch เพื่อให้ CPU มีงานทำระหว่างรอผลการตัดสินใจ แทนที่จะปล่อยให้ Pipeline ว่าง (Stall)

pipeline_02_fixed.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a;
6      int b[] = {80,70,60,50};
7      // int *c; // ไม่ได้ใช้ pointer *c เพื่อลด complexity
8      // c=b+2;
9
10     scanf("%d",&a);
11
12     // แก้ไข Control Hazard โดยลดความซับซ้อนของเงื่อนไข (Code Optimization)
13     // และจัดเรียงเงื่อนไขที่ "น่าจะเป็นจริงบ่อยที่สุด" ไว้บนสุด (Manual Prediction)
14
15     // สมมติว่าจะแนบส่วนใหญ่ผ่าน (A, B, C, D) เราย้ายมาเช็คก่อน
16     if (a >= 0 && a <= 100) {
17         if(a > 80) printf("A\n");
18         else if(a > b[1]) printf("B\n");
19         // else if(a > *c) printf("C\n"); // *c คือ b[2] = 60
20         else if(a > 60) printf("C\n");
21         else if(a >= b[3]) printf("D\n"); // b[3] คือ 50
22         else printf("F\n");
23     } else {
24         // กรณี Error (น้อยมากที่จะเกิด) เอาไว้ท้ายสุด
25         // ช่วยให้ Branch Prediction ทายถูกง่ายขึ้นว่า "มักจะเข้า if แรกเสมอ"
26         printf("Error\n");
27     }
28
29     return 0;
30 }
31
```

Pipeline 3

pipeline_03.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int i,j;
6      long sum,x;
7
8      for (i=0;i<10;i++){
9          for(j=0;j<10;j++){
10             x=i+j;
11             sum = sum+x;
12         }
13     }
14     printf("%ld\n",sum);
15 }
```

pipeline_03.s

```
51  .L3:
52      ldr r3, [fp, #-12]
53      cmp r3, #9
54      ble .L4
55      ldr r3, [fp, #-8]
56      add r3, r3, #1
57      str r3, [fp, #-8]
58  .L2:
59      ldr r3, [fp, #-8]
60      cmp r3, #9
61      ble .L5
62      ldr r1, [fp, #-16]
63      ldr r0, .L7
64      bl printf
65      mov r3, #0
66      mov r0, r3
67      sub sp, fp, #4
68      @ sp needed
69      pop {fp, lr}
70      bx lr
71  .L8:
72      .align 2
73  .L7:
74      .word .LC0
75      .size main, .-main
```

```
1      .cpu arm7tdmi
2      .arch armv4t
3      .fpu softvfp
4      .eabi_attribute 20, 1
5      .eabi_attribute 21, 1
6      .eabi_attribute 23, 3
7      .eabi_attribute 24, 1
8      .eabi_attribute 25, 1
9      .eabi_attribute 26, 1
10     .eabi_attribute 30, 6
11     .eabi_attribute 34, 0
12     .eabi_attribute 18, 4
13     .file "pipeline_03.c"
14     .text
15     .section .rodata
16     .align 2
17  .LC0:
18     .ascii "%ld\012\000"
19     .text
20     .align 2
21     .global main
22     .syntax unified
23     .arm
24     .type main, %function
25  main:
26     @ Function supports interworking.
27     @ args = 0, pretend = 0, frame = 16
28     @ frame_needed = 1, uses_anonymous_args = 0
29     push {fp, lr}
30     add fp, sp, #4
31     sub sp, sp, #16
32     mov r3, #0
33     str r3, [fp, #-8]
34     b .L2
35  .L5:
36     mov r3, #0
37     str r3, [fp, #-12]
38     b .L3
39  .L4:
40     ldr r2, [fp, #-8]
41     ldr r3, [fp, #-12]
42     add r3, r2, r3
43     str r3, [fp, #-20]
44     ldr r2, [fp, #-16]
45     ldr r3, [fp, #-20]
46     add r3, r2, r3
47     str r3, [fp, #-16]
48     ldr r3, [fp, #-12]
49     add r3, r3, #1
50     str r3, [fp, #-12]
```

ใน Pipeline 3 จะเกิดปัญหาร่วมกันระหว่าง **Data Hazard** และ **Control Hazard** ภายในโครงสร้าง Loop โดย **Data Hazard** เกิดจาก **Data Dependency** ของตัวแปร **sum** ที่ทำหน้าที่เป็น Accumulator ซึ่งมีการเขียนและอ่านค่าซ้ำๆ ในทุกรอบการวนลูป ($sum = sum + x$) ข้อมูลจากรอบก่อนหน้าอาจยังบันทึกไม่เสร็จเมื่อรอบใหม่ต้องการใช้ ทำให้เกิดโอกาส Stall ที่ขึ้น ส่วน **Control Hazard** เกิดจากการตรวจสอบเงื่อนไข Loop ($j < 10$) ในทุกรอบ ซึ่งถือเป็น Conditional Branch หากทำนายผิดก็จะต้องทำการ Flush Pipeline เช่นกัน

แนวทางการแก้ไขปัญหานี้ใน Loop คือการใช้ **Branch Prediction** ที่แม่นยำ (โดยธรรมชาติ Loop มักจะทำนายว่า Taken/วนลูปต่อ ได้ถูกต้องสูง) เพื่อลด Control Hazard และใช้วิธี **Loop Unrolling** (คลี่ลูป) เพื่อลดจำนวนครั้งที่ต้องตรวจสอบเงื่อนไขและกระโดด นอกจากนี้ การคลี่ลูปยังช่วยเปิดโอกาสให้ Compiler สามารถจัดเรียงคำสั่ง (**Instruction Scheduling**) ได้ดียิ่งขึ้น เพื่อลดผลกระทบจาก Data Hazard ภายใน Loop ได้อีกทางหนึ่ง

pipeline_03_fixed.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int i,j;
6      long sum = 0; // Initialize sum
7      long x1, x2, x3, x4, x5;
8
9      // แก้ไข Loop Hazard โดยใช้ Loop Unrolling (คลี่ลูป)
10     // ลดจำนวนครั้งที่ต้องเช็คเงื่อนไข (Branch) และลด Data Dependency ใน Loop
11
12     for (i=0;i<10;i++){
13         // Loop เดิมวน 10 รอบ -> คลี่ออกมาทำทีละ 5 รอบ (วนแค่ 2 รอบใหญ่)
14         for(j=0; j<10; j+=5){
15             // รอบที่ 1 (j)
16             x1 = i + j;
17             // รอบที่ 2 (j+1)
18             x2 = i + (j+1);
19             // รอบที่ 3 (j+2)
20             x3 = i + (j+2);
21             // รอบที่ 4 (j+3)
22             x4 = i + (j+3);
23             // รอบที่ 5 (j+4)
24             x5 = i + (j+4);
25
26             // นำผลรวมมารวมทีเดียวน ลดการอัปเดต sum ที่เกินไป (Accumulator Hazard)
27             sum = sum + x1 + x2 + x3 + x4 + x5;
28         }
29     }
30     printf("%ld\n",sum);
31 }
```